"QUICK PARK ASSIST- A Convenient Parking Spot Management System and EV Charging Solution"

A Project work

Submitted in Partial fulfillment for the award of

Graduate Degree of Bachelor of Technology (B.Tech.)

In

Computer Science & Engineering

(Session: 2024-25)

Submitted By

Swechchha Agrawal

(Enroll no: 0205CS211112)

Under the Guidance of

Prabhjyot kaur Haryal

(Computer Sc. & Engg.)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SHRI RAM INSTITUTE OF TECHNOLOGY, JABALPUR (M.P.)

RAJEEV GANDHI PROUDYOGIKI VISHWAVIDYALAYA, BHOPAL (M.P.)

SHRI RAM INSTITUTE OF TECHNOLOGY
Approved by AICTE New Delhi & Govt. of M.P.
(Affiliated to R.G.P.V.V. - University of Technology of Madhya Pradesh)
ISO 9001 : 2000 Certified Institution
Near ITI, MADHOTAL, JABALPUR - 482 002 (M.P.)
Ph. No. : 0761 - 2640291, 94 Fax No. : 2640294 Mobile - 9300104815

CERTIFICATE

This is to certify that

Swechchha Agrawal
(Enroll no: 0205CS211112)

students of 8th semester, Computer Sc. & Engg. S.R.I.T., Jabalpur have duly completed their final year project entitled ***"QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV Charging Solution"*** *for the partial fulfillment of the requirement for the degree of Bachelor of Technology as per R.G.P.V., Bhopal.*

They have successfully implemented and tested this project, which meets all the requirements specified under my guidance.

Prabhjyot kaur Haryal	*Prof. Brajesh Patel.*	*Dr. Shailesh Gupta*
(Project Guide)	*(H.O.D.)*	*(Principal)*
Computer Sc & Engg. dept., SRIT	*Computer Sc. & Engg. dept., SRIT*	*SRIT, Jabalpur*

SHRI RAM INSTITUTE OF TECHNOLOGY
Approved by AICTE New Delhi & Govt. of M.P.
(Affiliated to R.G.P.V.V. - University of Technology of Madhya Pradesh)
ISO 9001 : 2000 Certified Institution
Near ITI, MADHOTAL, JABALPUR - 482 002 (M.P.)
Ph. No. : 0761 - 2640291, 94 Fax No. : 2640294 Mobile - 9300104815

CERTIFICATE

This is to certify that

Swechchha Agrawal
(Enroll no: 0205CS211112)

students of 8th semester, Computer Sc. & Engg. S.R.I.T., Jabalpur have duly completed their final year project entitled ***"QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV Charging Solution"*** *for the partial fulfillment of the requirement for the degree of Bachelor of Technology as per R.G.P.V., Bhopal.*

They have successfully implemented and tested this project, which meets all the requirements.

INTERNAL EXAMINER

Date:

EXTERNAL EXAMINER

Date:

ACKNOWLEDGEMENT

First of all, we thank the ***GOD*** who is constantly showering his blessing and love on us.

As one self can accomplish nothing, this project is also not an exception. I would like to have some space to acknowledge some of them that frequently fade in to the background during the course of our project work; we got help, advices, suggestions and co-operation from many people. Some influenced us, some inspired us, and some helped us in completing our project work.

It is our great privilege; pride and honor in expressing our deepest sense of gratitude to my esteemed inspiring, mentor **Prabhjyot kaur Haryal** in presenting this project is entitled "**QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV Charging Solution**", his constant inspiration, memorable guidance, innovative ideas, at various stages and above all her untiring valuable support has brought out this project to an exquisite workmanship and great success.

We express my profound and sincere gratitude to Mr. R. Karsoliya (Chairman) Shri Ram Institute of Technology, Jabalpur (M.P.), affiliated to Rajiv Gandhi Technical University, Bhopal, (M.P.) for providing facilities needed in connection with our project work.

We express our gratitude to Dr. Shailesh Gupta Principal, Dr. Ruchi K. Patel, Prof. Brajesh Patel, Prof. Rajendra Arakh, Prof. Deepak Singh Rajput, Prof. Apoorv Khare, Prof Richa Shukla for their support and help we received from him throughout this project work and encouragement to carry out this project.

Swechchha Agrawal
(Enroll no: 0205CS211112)

SHRI RAM INSTITUTE OF TECHNOLOGY
Approved by AICTE New Delhi & Govt. of M.P.
(Affiliated to R.G.P.V.V. - University of Technology of Madhya Pradesh)
ISO 9001 : 2000 Certified Institution
Near ITI, MADHOTAL, JABALPUR - 482 002 (M.P.)
Ph. No. : 0761 - 2640291, 94 Fax No. : 2640294 Mobile - 9300104815

DECLARATION

We hereby declare that the project entitled **" QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV Charging Solution"** submitted in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Science and Engineering** is a **record of our original work** carried out under the guidance of **Prabhjyot kaur Haryal** at **Shri Ram Institute of Technology, Jabalpur**.

This project has not been submitted previously, in part or full, for the award of any other degree or diploma in any institute or university. The information and data presented in this report are true to the best of our knowledge and belief. Any reference to previously published work has been duly acknowledged and cited in the references.
We take full responsibility for the integrity and authenticity of the contents of this project.

Swechchha Agrawal
(Enroll no: 0205CS211112)

PREFACE

In the era of rapid urbanization and increasing vehicle usage, the challenge of finding convenient and efficient parking has become a significant concern for city dwellers and commuters. Our project '**QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV Charging Solution'** is an innovative attempt to address this issue through a smart, tech-driven application that combines real-time parking availability with EV charging and additional vehicle services.

This report encapsulates the development journey of the Quick Park Assist system, which is designed to streamline the parking process for users while offering flexibility and control to parking spot owners. By incorporating modern technologies such as Spring Boot, MySQL, Splunk, and Prometheus–Grafana for monitoring, along with secure user authentication, we aimed to deliver a reliable and user-friendly platform.

We express our heartfelt gratitude to our mentors, institution, and all contributors who supported and guided us throughout the development of this project. It is our sincere hope that this work will serve as a foundation for future innovations in the field of smart waste management and sustainable development.

Swechchha Agrawal
(Enroll no: 0205CS211112)

INDEX

1. INTRODUCTION

With the rapid increase in the number of vehicles and the growth of urban areas, parking has become a major issue for both drivers and city planners. Finding an available parking spot, especially during peak hours, often leads to traffic congestion, increased fuel consumption, and unnecessary stress. The problem becomes even more complex with the rising demand for EV (Electric Vehicle) charging stations and the need for well-maintained parking infrastructure.

To address these challenges, we have developed **QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV charging solution**, a smart and user- friendly web-based application designed to simplify parking and vehicle service management. This system allows users to search for nearby parking spots in real-time, reserve them in advance, and access additional services such as EV charging, car cleaning, and maintenance. It also empowers parking spot owners to list, update, and manage their parking spaces efficiently.

The application supports two types of users: **Vehicle Owners** and **Parking Spot Owners**, with role-based access to different features. It includes secure registration and login using Spring Security, and ensures data safety and performance with the help of MySQL, Splunk (for log monitoring), SonarQube (for code analysis), and Prometheus–Grafana (for performance metrics).

Quick Park Assist is developed using Java, Spring Boot for the backend, and Thymeleaf, HTML, CSS, and JavaScript for the front end. The system is designed to be scalable, maintainable, and ready for future enhancements like AI-based dynamic pricing, real-time parking updates via IoT sensors, and a dedicated mobile application.

Through this project, our goal is to create a smart solution that not only helps drivers save time and effort but also supports better parking space utilization, contributing to smarter and cleaner cities.

2. ANALYSIS

The **QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV charging solution** project is designed to address the growing problem of parking in urban areas by providing a smart, user-friendly platform for both vehicle owners and parking spot providers. It offers features such as real-time parking spot booking, EV charging slot reservations, and access to add-on services like car cleaning and maintenance. The system is built using modern technologies like Java, Spring Boot, MySQL, and Spring Security, ensuring a secure and efficient experience. Its layered architecture separates the user interface, business logic, and data storage, making the application scalable and maintainable. With tools like Prometheus, Grafana, Splunk, and SonarQube integrated for monitoring and code quality, the project maintains high standards in performance and security. While it offers significant benefits such as time- saving, improved space utilization, and enhanced user convenience, it currently lacks a dedicated mobile app and real-time IoT integration for live parking updates. Overall, Quick Park Assist presents a practical and scalable solution to modern parking challenges and lays a strong foundation for future enhancements like AI-based pricing and smart city integration.

2.1 OBJECTIVE

The objective of a **QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV Charging Solution** system is to assist drivers in easily parking their vehicles in tight or crowded spaces by providing automated support. The system typically uses sensors, cameras, or other technologies to detect the available parking space, and it either provides guidance (like visual or audio cues) or takes over the control of the vehicle (e.g., steering, braking, and acceleration) to park the car without requiring much driver input. The key goals are:

1. **Ease of Parking:** Simplifying the parking process, especially in tight or complex spaces.
2. **Safety:** Reducing the risk of accidents or damage to the vehicle during parking.
3. **Time Efficiency:** Allowing quicker and more efficient parking, saving time and reducing stress for the driver.
4. **Convenience:** Making parking less of a hassle, especially in busy urban areas or parking garages.

2.2 REQUIREMENT GATHERING

🔍 1. Functional Requirements (What the system should do)

👤 User Registration & Authentication

- Users should be able to register as:
  - Vehicle Owner
  - Parking Spot Owner
- Login using email and password
- OTP verification for secure actions
- Role-based access (RBAC)

🅿️ Parking Spot Management

- Parking spot owners can:
  - Add new parking spots (with location, type, availability, and price)
  - Update or delete existing parking spots
- All users can:
  - Search for available parking spots by location and time
  - View parking spot details

📅 Booking Management

- Users can:
  - Book available parking spots
  - Modify or cancel their bookings
  - View current and past bookings
  - Check bookings using registered mobile number or spot name

🩹 Addon Services

- Parking spot owners can:
  - Add extra services (e.g., car wash, polishing)
  - Update or delete services
- Users can:
  - View and book addon services during parking reservation

⚡ EV Charging Reservation

- Users can:
  - Book EV charging slots based on availability
  - View, update, or cancel charging reservations

⚙️ 2. Non-Functional Requirements (How the system should perform)

Requirement	Description
Performance	The application should respond quickly and handle multiple users/bookings at once.
Security	Use secure login with password encryption and OTP. Implement role-based access.
Usability	The interface should be clean, responsive, and easy to use for all types of users.
Scalability	The system should scale to support more users and spots in the future.
Maintainability	Code should be modular and easy to update or debug.
Availability	The application should be available 24/7 with minimal downtime.

🛠️ 3. Technical Requirements

Layer	Technologies Used
Frontend	HTML, CSS, JavaScript, Thymeleaf
Backend	Java, Spring Boot (REST APIs)
Authentication	Spring Security
Database	MySQL
API Testing	Postman
Documentation	Swagger
Monitoring & Logging	Splunk, Prometheus, Grafana
Code Quality	SonarQube

2.3 HARDWARE REQUIREMENT

1. Client-Side Requirements (User's Device)

Component	Minimum Requirement	Recommended Specification
Processor	Dual Core 1.8 GHz	Quad Core 2.5 GHz or higher
RAM	2 GB	4 GB or higher
Storage	500 MB of free space	1 GB or more
Display	1024×768 resolution	1366×768 or higher
Internet	1 Mbps internet connection	4 Mbps or higher
Browser	Chrome, Firefox, Edge (latest versions)	Any modern, updated browser

2. Server-Side Requirements (Hosting Environment)

Component	Minimum Requirement	Recommended Specification
Processor	Quad Core 2.4 GHz	Octa Core or higher
RAM	8 GB	16 GB or higher
Storage	100 GB HDD or SSD	250 GB SSD or higher
Network	100 Mbps Ethernet/Fiber	1 Gbps high-speed connection
Operating System	Linux (Ubuntu/CentOS) or Windows Server	Latest LTS version
Database Server	MySQL 5.7 or above	MySQL 8.0 or higher

2.4 SOFTWARE REQUIREMENT

- **Operating System**: Windows 10
- **Backend**: Java, Spring Boot
- **Frontend:** Thymeleaf, HTML, CSS, JavaScript
- **Database:** MySQL
- **Other Tools:** Postman, Swagger, SonarQube
- Splunk, Prometheus, Grafana

2.5 Feasibility Study

The **Feasibility Study** evaluates whether the "Quick Park Assist" is practical and viable for real-world implementation. The feasibility study determines whether the proposed system is viable from technical, operational, and economic perspectives:

1. Technical Feasibility

The project is technically feasible as it utilizes well-established technologies such as Java, Spring Boot, MySQL, and REST APIs. The architecture supports scalability and can be integrated with IoT and mobile platforms in the future.

2. Operational Feasibility

The system is designed to be user-friendly, with clear role-based access for both vehicle owners and parking spot owners. The application streamlines real-life operations like booking, listing, and EV charging, ensuring smooth daily use.

3. Economic Feasibility

The application is cost-effective to develop using open-source tools and frameworks. Hosting and database requirements are minimal for initial deployment, reducing operational costs. Future monetization (e.g., dynamic pricing, add-on services) can generate revenue.

4. Legal & Ethical Feasibility

No sensitive data is misused. Proper authentication (via OTP/email verification) and secure login ensure user privacy. GDPR and other local digital security standards can be adhered to if required.

2.6 Software Model

The development of "Quick Park Assist" follows the Agile Software Development Model.

Agile Model Highlights:

- **Iterative Development**: The project was developed in multiple small, manageable iterations, allowing for continuous improvements based on feedback.
- **Customer Involvement**: Regular interactions and feedback were considered to enhance the usability and efficiency of the application.
- **Flexible Design**: Enhancements such as dynamic pricing and IoT integration are easily adoptable in future iterations.
- **Early Testing**: Functional modules like user login, parking booking, and EV reservations were tested early and improved continuously.

Why Agile?

- Supports dynamic requirements.
- Encourages rapid development and deployment.
- Enhances collaboration and adaptability.
- Provides better quality through frequent testing and integration.

2.7 SOFTWARE DEVELOPMENT LIFE CYCLE

1. Requirement Analysis

- Identified the core features such as user registration, parking spot management, booking system, EV charging, and addon services.
- Requirements were refined through team discussions and mock user feedback.

2. System Design

- **Hardware Design**:
  - Standard desktop used by developers with-Intel Core i3/i5 processors, 4–8 GB RAM, Internet access for collaboration and API testing.
  - Power supply planning 5V USB adapter or portable battery unit.
  - Mounting Mechanism: Sensor setup on individual parking slots to detect occupancy
  - Wi-Fi Module: For real-time data transmission to the backend
- **Software Architecture**:
  - Built using HTML, CSS, JavaScript, and Thymeleaf templates.
  - Sends requests to the backend using REST APIs.
  - implemented in Java using the Spring Boot framework.
  - Uses MySQL to store data such as user info, vehicle details, parking spot data, bookings, services, and more.
- **Data Flow Design**:
  [User] --> (Register/Login) --> [System] --> (Store in) --> [User DB]
  [Owner] --> (Add Spot) --> [System] --> (Update) --> [Spot DB]
  [Vehicle Owner] --> (Book Spot) --> [System] --> (Update) --> [Booking DB]

3. Implementation / Development

- **Frontend Development**
  Technologies Used: HTML, CSS, JavaScript, Thymeleaf
  - User registration and login interfaces
  - Dashboard for users to view and manage bookings
  - Responsive design for accessibility on various devices
- **Backend Development**
  - Framework: Spring Boot (Java)
  - Controllers: Handle HTTP requests and responses
  - Services: Contain business logic for processing data
  - Repositories: Manage data persistence using Spring Data JPA
- **Database Integration**
  - Database: MySQL

- Tables for users, parking spots, bookings, vehicles, and services

- **API Development and Testing**
  - **RESTful APIs**: Developed for all major functionalities, including user management, parking spot operations, and booking processes
  - **Testing Tools**:
    - **Postman**: Used for manual testing of API endpoints
    - **Swagger**: Integrated for API documentation and testing
- **Deployment and Monitoring**
  - **Deployment**: Application deployed on a local server for testing purposes
  - **Monitoring Tools**:
    - **Prometheus**: Set up for collecting metrics (planned for future integration)
    - **Grafana**: Configured for visualizing system performance metrics (planned for future integration)

4. Testing

- **Unit Testing**
  - Objective: To verify the functionality of individual components or modules.
  - Approach: Each function, such as user registration, login, parking spot addition, and booking, was tested in isolation.
  - Tools Used: JUnit for Java-based unit tests.
- **Integration Testing**
  - Objective: To ensure that different modules or services used by the application interact correctly.
  - Approach: Tested the interaction between the frontend and backend, ensuring data flows correctly through APIs.
  - Tools Used: Postman for API testing.
- **System Testing**
  - Objective: To validate the complete and integrated application against the requirements.
  - Approach: Conducted end-to-end testing scenarios, including user workflows like searching for parking spots, booking, and managing reservations.
  - Tools Used: Selenium for automated browser testing.
- **Security Testing**
  - Objective: To identify vulnerabilities and ensure data protection.
  - Approach: Tested for common security issues such as SQL injection, cross-site scripting (XSS), and ensured secure data transmission.
  - Tools Used: OWASP ZAP for vulnerability scanning.

5. Deployment

- The system was deployed locally for testing using Spring Boot's embedded server.
- Set up the MySQL database with appropriate schemas and initial data.
- Implemented logging mechanisms to track application behavior and errors.

- Optional tools like Prometheus and Grafana integrated for monitoring.

6. Maintenance & Updates

- Regular system updates (bug fixes, feature upgrades).
- Modifies the software to remain compatible with evolving environments, such as new operating systems or hardware.
- Improves performance and user experience by refining features and interfaces.
- Applies security patches to address vulnerabilities and protect user data.
- Optional tools like Prometheus and Grafana integrated for monitoring.

7. Maintenance & Updates

- Regular system updates (bug fixes, feature upgrades).
- Modifies the software to remain compatible with evolving environments, such as new operating systems or hardware.
- Improves performance and user experience by refining features and interfaces.
- Applies security patches to address vulnerabilities and protect user data.

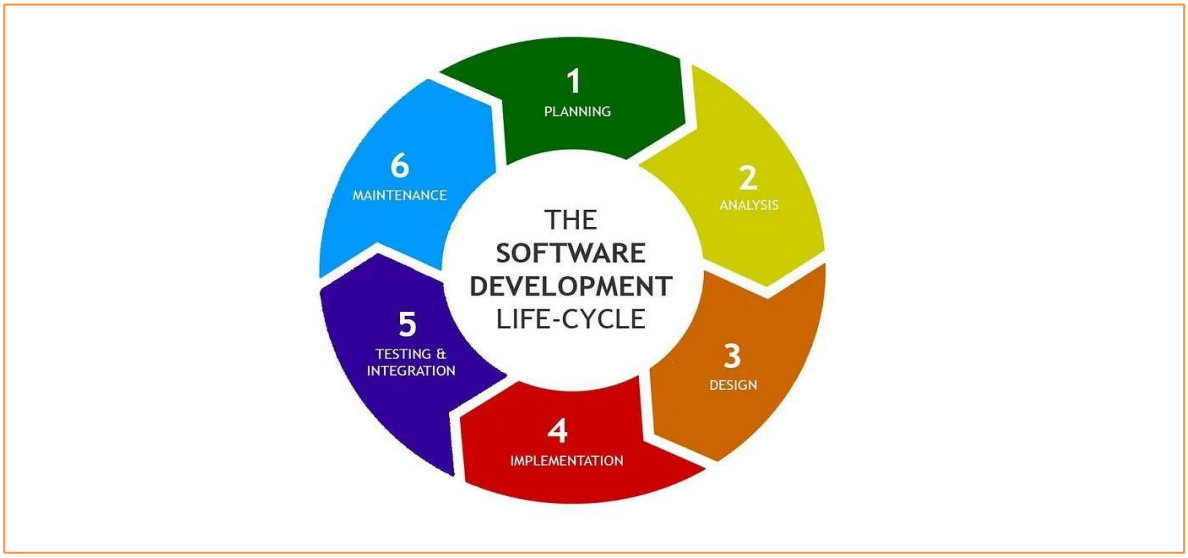


Figure 1: Software Development Life- Cycle

2.8 COST ESTIMATION

🔧 **Assume Cost Estimation for " QUICK PARK ASSIST – A Convenient Parking Spot Management System and EV charging solution " Project (INR)**

1. **Team:** 3 students (academic work), ₹2,500/hour for professional estimate
2. **Hardware:** Minimal, student-owned laptops + optional IoT sensors
3. **Software:** Mostly open-source tools
4. **Hosting:** Local is free; cloud estimated
5. **Duration:** 6–8 months
6. **Currency:** INR, localized pricing as of April 2025
7. **Enhancements:** IoT, mobile app, AI integration (optional future scope)

📊 **Cost Breakdown**

Category	Item	Details	Estimated Cost (INR)
Development	Developer Effort (3 developers)	3 × 500 hrs @ ₹2,500/hr (hypothetical)	₹37,50,000 (Approx.)
	Project Guidance	Faculty support (academic)	₹0
Hardware	Developer Laptops	Existing student machines	₹0
	IoT Sensors (future)	10 × ₹800 = Ultrasonic, ESP8266	₹8,000 (Approx.)
	Power & Connectivity	Adapters, Wi-Fi modules	₹1,500(Approx.)
Software/Tools	IDEs (IntelliJ, VS Code)	Free (Community versions)	₹0
	Spring Boot, Thymeleaf, MySQL	Open-source	₹0
	Splunk	Free/academic tier	₹0
	SonarQube, Prometheus, Grafana	Open-source	₹0
	Postman, Swagger	Free tier	₹0
Testing	Unit/Integration (JUnit, Mockito)	Included in dev effort	₹0
	Security Testing (OWASP ZAP)	Open-source	₹0
	Performance (Prometheus, Grafana)	Open-source	₹0
Deployment	Local Deployment	Free using embedded server	₹0
	Cloud Hosting (Production)	₹800/month × 12 months	₹9,600(Approx.)
	Domain Name	Annual registration cost	₹1,200(Approx.)
Maintenance	Bug Fixes/Updates	~50 hrs @ ₹2,500/hr	₹1,25,000(Approx.)
Future Enhancements	Mobile App (Android/iOS)	~300 hrs @ ₹2,500/hr	₹7,50,000(Approx.)
	AI-Based Dynamic Pricing	~200 hrs @ ₹2,500/hr + compute (₹8,000)	₹5,08,000(Approx.)
	IoT Integration	Hardware ₹40,000 + 100 hrs dev	₹2,90,000(Approx.)

Miscellaneous	Docs & Presentation	Printing, stationery	₹4,000(Approx.)

📌 **Summary Table**

Project Type	Total Estimated Cost (INR)
Academic Project	₹24,300(Approx.)
Professional Estimate	₹54,47,300(Approx.)

Notes:

- Academic project excludes labor cost; most tools and resources are free.
- Professional estimate includes real-world development rates and advanced features.
- Future features like AI and IoT integration are optional and module.

3. DESIGN

The system design of the **Quick Park Assist** application follows a modular and layered architecture, ensuring scalability, maintainability, and performance

Architecture Overview

The application follows a three-tier architecture consisting of:

- Presentation Layer (Frontend)
  Built using HTML, CSS, JavaScript, and Thymeleaf, this layer provides an intuitive interface for users to interact with the system.
- Business Logic Layer (Backend)
  Developed in Java using Spring Boot, this layer handles application logic, user authentication, role-based access, and business operations.
- Data Layer (Database and Logging)
  All data is stored in a MySQL database. Logging and monitoring are handled via Splunk, Prometheus, and Grafana.

3.1 Input Requirement

1. User Registration

- Name, Email (OTP), Phone, Address, Password, Role (Vehicle/Parking Owner)

2. Login

- Email, Password (with forgot/reset via OTP)

3. Vehicle Management

- Vehicle Number, Type (2W/4W/EV), Model

4. Parking Spot Management

- Spot ID, Location, Type (EV/Indoor/Outdoor), Price, Availability, Instructions

5. Booking System

- Search Location, Date & Time, Spot Selection, Vehicle, Duration

6. Add-on Services

- Service Name, Description, Price, Duration (Owner inputs)
- Service Selection (User inputs)

7. EV Charging Reservation

- Station Selection, Date & Time, Vehicle, Duration

8. Account/Profile Management

- Profile Updates, Password Change, Account Deactivation/Deletion

3.2 Data Flow Diagram / Object diagram

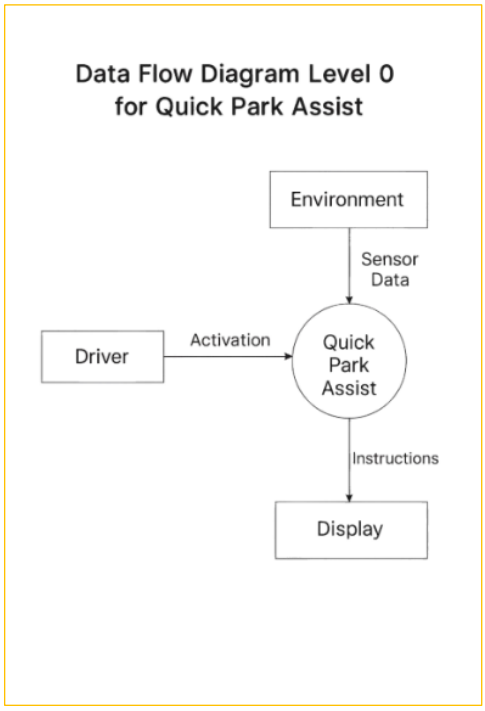


Figure 2: Level 0 DFD

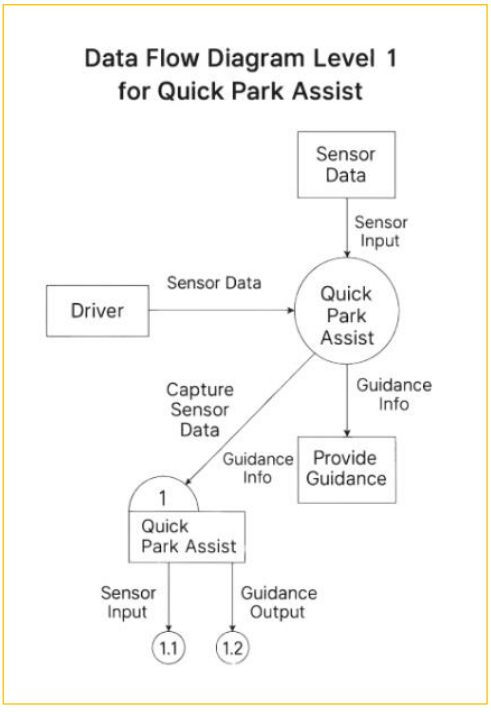


Figure 3: Level 1 DFD

3.3 Control Flow Diagram

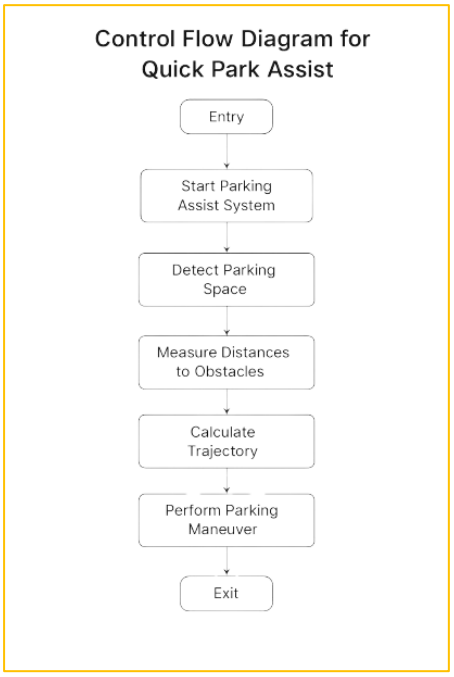


Figure 4 : Control flow diagram

4. TOOLS AND TECHNOLOGY USED

1. **Frontend / UI Technologies**
   - Thymeleaf: (version 3.1.3.Release) A modern server-side Java template engine used for rendering dynamic web pages in the Spring Boot environment.
   - HTML, CSS, JavaScript: Core technologies for structuring, styling, and adding interactivity to the web interface.

2. **Backend Technologies**
   - Java:(version 17) The primary programming language used for developing the server-side logic.
   - Spring Boot: (version 3.4.2) A framework for building production-ready RESTful web services quickly. It simplifies the setup and configuration process.
   - Spring Security: Provides authentication and authorization capabilities, ensuring secure login and role-based access control.

3. **API Documentation & Testing**
   - Swagger: Automatically generates interactive API documentation, allowing developers and testers to explore and test endpoints easily.
   - Postman: A popular tool used for testing APIs by sending requests and validating responses during development.

4. **Database & Logging**
   - MySQL: A relational database used to store user, parking, vehicle, booking, and service-related data.
   - Splunk: A tool for collecting, indexing, and analyzing log data. It helps monitor the health and activity of the application.

5. **Code Quality & Monitoring**
   - SonarQube: An open-source platform for continuous inspection of code quality, detecting bugs, vulnerabilities, and code smells.
   - Prometheus: A metrics collection system used for monitoring application performance.
   - Grafana: A data visualization tool that works with Prometheus to create dashboards and graphs for real-time monitoring.

6. **Modules Integrated with Technology Stack**
   - User Registration: Secure onboarding with Spring Security, email verification, and profile management.
   - Parking Spot Management: Backend services (Spring Boot + MySQL) allow spot

owners to manage availability and pricing.

- Booking Management: Java controllers handle bookings, integrated with UI and database for real-time updates.
- Add-on Services: Dynamic integration of additional vehicle services, managed via backend and reflected in the UI.
- EV Charging Reservation: Real-time slot booking using REST APIs with user-friendly interface.

7. Future Enhancements (Proposed Technologies)

- AI-Based Dynamic Pricing: Implemented using AI/ML algorithms to optimize rates based on time, demand, and location.
- Mobile Application: Developed using Android/iOS frameworks for accessibility and real-time push notifications.
- IoT-Based Parking Availability: Incorporating sensors and microcontrollers (e.g., ESP32, Raspberry Pi) to detect live parking occupancy.

5. CODING

Frontend Code

Home Page Module

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Home</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
<link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css"
rel="stylesheet">
<!--CSS LINKS-->
<link rel="stylesheet"  type="text/css" th:href="@{/css/HomeStyle.css}">
<link rel="stylesheet"  type="text/css" href="./css/HomeStyle.css">
</head>
<body>
<header  th:style="'background: linear-gradient( rgb(0 14 38  / 70%), rgba(6 26 79 / 48%)),
url('+ @{/images/bgImage.jpg }+') no-repeat center center/cover; background-attachment:fixed;'">
<div class="container-fluid">
<nav class="navbar navbar-expand-lg navbar-light">
<a class="navbar-brand text-light" href="#">Smart Parking Booking</a>
<button class="navbar-toggler custom-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation"
th:style="'background: linear-gradient(to right,  #5f89ff, #9d7bfe)  no-repeat;' ">
<span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarNav">
<ul class="navbar-nav ms-auto">
<li class="nav-item">
<a class="nav-link text-dark" th:href="@{/}" aria-current="page">HOME</a>
</li>

<li class="nav-item">
<a class="nav-link text-dark" th:href="@{/dashboard}">DASHBOARD</a>
</li>

<li class="nav-item">
<a class="nav-link text-dark" th:href="@{/smart-spots/add-spot}">SMARTSPOTS</a>
</li>
<li class="nav-item" th:if="${session.userType == 'VEHICLE_OWNER'}">
<a class="nav-link text-dark" th:href="@{/bookingSpot/}">BOOKING</a>
</li>
<li class="nav-item">
<a class="nav-link text-dark" th:href="@{/addon/new}">ADDON SERVICES</a>
</li>
```

```
<div class="testimonials">
<div class="testimonial">
<p>
"I love how convenient Smart Parking Spots is! I can easily find parking near important events, even during busy hours. The booking process is seamless, and I don’t have to deal with paying meters or machines anymore. It’s stress-free, and I’m always on time. Definitely my go-to app for parking!"
</p>
<div class="client-info">
<img th:src="@{/images/dave.jpg}" alt="Dave M">
<div class="client-details">
<h4>Dave M</h4>
<p>Commuter</p>
</div>
</div>
</div>
<div class="testimonial">
<p>
"Smart Parking Spots has been a game-changer for me! I used to spend so much time searching for a parking spot. Now, I just book in advance and go straight in. It’s quick, easy, and reliable. No more worrying about parking again. Highly recommend it to anyone who hates the parking struggle."
</p>
<div class="client-info">
<img th:src="@{/images/hannah.jpg}" alt="Hannah T">
<div class="client-details">
<h4>Hannah T</h4>
<p>Event-Goer</p>
</div>
</div>
</div>
<div class="testimonial">
<p>
"The app has simplified parking for me. I love the easy interface and reliable service. It’s especially helpful when attending crowded events, saving me so much time and stress."
</p>
<div class="client-info">
<img th:src="@{/images/ben.jpg}" alt="Ben R">
<div class="client-details">
<h4>Ben R</h4>
<p>Business Traveler</p>
</div>
</div>
</div>
<div class="testimonial">
<p>
"I’ve never experienced such a smooth parking process before. From booking to payment, everything is effortless. Highly recommend this app to anyone who values convenience."
</p>
<div class="client-info">
<img th:src="@{/images/steve.jpg}" alt="Steve L">
<div class="client-details">
<h4>Steve L</h4>
<p>Frequent User</p>
```

```
}

// Add event listeners to navigation dots
navDots.forEach((dot, index) => {
dot.addEventListener('click', () => {
currentSlideIndex = index;
updateTestimonials(currentSlideIndex);
});
});

// Auto-cycle testimonials (optional)
setInterval(() => {
currentSlideIndex = (currentSlideIndex + 1) % 2; // Only two sets of testimonials (0 and 1)
updateTestimonials(currentSlideIndex);
}, 5000); // Change every 5 seconds

</script>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"></script>

</body>
</html>
```

Login Module

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">

<head>
<!-- Required meta tags -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
<!-- FAVICON ICONS-->
<link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css"
rel="stylesheet">
<!-- CSS file -->
<link rel="stylesheet"  type="text/css" th:href="@{/css/style.css}">
<title>Smart Parking - yourWebPageName</title>
</head>
<body th:style="'background: linear-gradient( rgb(0 14 38  / 70%), rgba(6 26 79 / 48%)),
url('+ @{/images/bgImage.jpg }+') no-repeat center center/cover; background-attachment:fixed;'">
<div class="container-fluid">
<nav class="navbar navbar-expand-lg navbar-light">
<a class="navbar-brand text-light" href="#">Smart Parking Booking</a>
<button class="navbar-toggler custom-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation"
th:style="'background: linear-gradient(to right,  #5f89ff, #9d7bfe)  no-repeat;' ">
<span class="navbar-toggler-icon"></span>
</button>
<!-- Booking Form Container -->
```

```
<div class="container d-flex flex-row justify-content-center align-items-center" style="border-radius:5px;max-width:550px;padding:5px;min-height:90vh">
<!-- Form Section -->
<div class="card">
<div class="card-body w-100">
<div class="header-blue">
<h3 class="text-center bg-primary text-light" style="padding:10px; border-radius:5px">Login to your Account</h3>
</div>
<div class="login-header">
<p class="welcome-text text-center">Welcome back to SmartPark! Please enter your credentials.</p>
</div>
<div th:if="${param.error}" class="alert alert-danger">
Invalid email or password.
</div>
<div th:if="${successMessage}" class="alert alert-success" th:text="${successMessage}"></div>

<form th:action="@{/user-login}" method="post">
<div class="form-group mb-3">
<label for="email">Email Address</label>
<input type="email" class="form-control" id="email" name="email" placeholder="Enter your email" required>
</div>

<div class="form-group mb-3">
<div class="d-flex justify-content-between mb-1">
<label for="password">Password</label>
</div>
<input type="password" class="form-control" id="password" name="password" placeholder="Enter your password" required><br>
<a th:href="@{/auth/forgot}" class="text-primary">Forgot Password?</a>
</div>

<button type="submit" class="btn  btn-success btn-block w-100">Login</button>
</form>

<div class="mt-3">
<a th:href="@{/register}" class="text-primary">New User?</a>
</div>
</div>
</div>
</div>
<footer class="text-center">
&copy; 2024 All Rights Reserved by Smart Parking Spots
</footer>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"></script>

</body>
</html>
```

Dashboard

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">

<head>
<!-- Required meta tags -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

<!-- Bootstrap CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
<!-- FAVICON ICONS-->
<link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css"
rel="stylesheet">
<!-- CSS file -->
<link rel="stylesheet"  type="text/css" th:href="@{/css/style.css}">
<title>Parking Spot Booking</title>
</head>
<body th:style="'background: linear-gradient( rgb(0 14 38  / 70%), rgba(6 26 79 / 48%)),
url('+ @{/images/bgImage.jpg }+') no-repeat center center/cover; background-attachment:fixed;'">
<div class="container-fluid">
<nav class="navbar navbar-expand-lg navbar-light">
<a class="navbar-brand text-light" href="#">Smart Parking Booking</a>
<button class="navbar-toggler custom-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation"
th:style="'background: linear-gradient(to right,  #5f89ff, #9d7bfe)  no-repeat;' ">
<span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarNav">
<ul class="navbar-nav ms-auto">
<li class="nav-item">
<a class="nav-link text-dark" th:href="@{/}" aria-current="page">HOME</a>
</li>
<li class="nav-item">
<a class="nav-link text-dark" th:href="@{/dashboard}">DASHBOARD</a>
</li>

<!-- Show SMARTSPOTS only for Spot Owners -->
<li class="nav-item">
<a class="nav-link text-dark" th:href="@{/smart-spots/add-spot}">SMARTSPOTS</a>
</li>

<!-- Show these items only for Vehicle Owners -->
<li class="nav-item" th:if="${session.userType == 'VEHICLE_OWNER'}">
<a class="nav-link text-dark" th:href="@{/bookingSpot/}">BOOKING</a>
</li>
<li class="nav-item">
<a class="nav-link text-dark" th:href="@{/addon/new}">ADDON SERVICES</a>
</li>
<li class="nav-item" th:if="${session.userType == 'VEHICLE_OWNER'}">
```

```
<a class="nav-link text-dark" th:href="@{/ev-charging/add}">EV CHARGING</a>
</li>
<li class="nav-item">
<a th:if="${session.userIsThere == true}" class="nav-link text-dark"
th:href="@{/logout}">LOGOUT</a>
</li>
</ul>
</div>
</nav>
</div>
<!-- Booking Form Container -->

<div class="container" style="border-radius:5px;max-width:950px; padding:10px;">
<div class="main-row" style="border-radius:5px;s">
<!-- Menu Section -->
<div class="col-md-3 sidebar">
<h2 class="text-light">Menu</h2>
<div class="menu-btns">
<a th:href="@{/dashboard}" class="fw-bold fst-italic">OverView</a>
<a th:href="@{/profile/edit}">Update Profile</a>

<!-- Show only for Vehicle Owners -->
<div th:if="${session.userType == 'VEHICLE_OWNER'}">
<a th:href="@{/vehicles/add}">Add Vehicle</a>
<a th:href="@{/vehicles/editVehicle}">Manage Vehicles</a>
</div>

<!-- Show only for Spot Owners -->
<div th:if="${session.userType == 'SPOT_OWNER'}">
<a th:href="@{/vehicles/search}">View User by Vehicle Number</a>
</div>

<a th:href="@{/auth/change-password}">Change Password</a>
<a th:href="@{/profileAction}">Delete Account</a>
<a th:href="@{/logout}">Logout</a>
</div>
</div>

<!-- Form Section -->
<div class="col-md-9 bg-light formSection d-flex flex-column">
<div class="card w-100">
<h3 class="text-center bg-primary text-light w-100" style="padding:10px; border-radius:5px">User
Profile Overview</h3>
<!-- Alert Messages -->
<div th:if="${successMessage}" class="alert alert-success" th:text="${successMessage}"></div>
<div th:if="${errorMessage}" class="alert alert-danger" th:text="${errorMessage}"></div>
<!-- Stats for Vehicle Owners -->
<div th:if="${session.userType == 'VEHICLE_OWNER'}"
class="d-flex justify-content-center align-items-center gap-3">
<div>
<div class="stats-card text-center p-3">
<i class="fas fa-parking icon-large text-primary"></i>
```

```
</div>
<div class="card w-100 p-2">
<div class="card-header">
<h5 class="card-title font-weight-bold">Recent Activity</h5>
</div>
<div class="card-body">
<div class="list-group" id="recentActivity">
</div>
</div>
</div>
</div>
</div>
</div>
</div>
<footer class="text-center">
&copy; 2024 All Rights Reserved by Smart Parking Spots
</footer>
<div class="container" style="border-radius:5px;max-width:950px; padding:10px;">
<div class="main-row" style="border-radius:5px;s">
<!-- Menu Section -->
<div class="col-md-3 sidebar">
<h2 class="text-light">Menu</h2>
<div class="menu-btns">
<a th:href="@{/dashboard}" class="fw-bold fst-italic">OverView</a>
<a th:href="@{/profile/edit}">Update Profile</a>

<!-- Show only for Vehicle Owners -->
<div th:if="${session.userType == 'VEHICLE_OWNER'}">
<a th:href="@{/vehicles/add}">Add Vehicle</a>
<a th:href="@{/vehicles/editVehicle}">Manage Vehicles</a>
</div>

<!-- Show only for Spot Owners -->
<div th:if="${session.userType == 'SPOT_OWNER'}">
<a th:href="@{/vehicles/search}">View User by Vehicle Number</a>
</div>

<a th:href="@{/auth/change-password}">Change Password</a>
<a th:href="@{/profileAction}">Delete Account</a>
<a th:href="@{/logout}">Logout</a>
</div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"></script>

<script>
async function fetchStats() {
try {
const response = await fetch('/api/stats');
if (!response.ok) throw new Error('Failed to fetch stats');
const stats = await response.json();
if (stats.totalSpots !== undefined) {  // Ensure it's for SPOT_OWNER
```

```
document.getElementById('totalSpots').textContent = stats.totalSpots || 0;
document.getElementById('currentBookings').textContent = stats.currentBookings || 0;
document.getElementById('totalRevenue').textContent = `$${stats.totalRevenue || 0}`;
} else {  // VEHICLE_OWNER
document.getElementById('availableSpots').textContent = stats.availableSpots || 0;
document.getElementById('activeBookings').textContent = stats.activeBookings || 0;
document.getElementById('totalHours').textContent = stats.totalHours || 0;
document.getElementById('amountSpent').textContent = `$${stats.amountSpent || 0}`;
}
} catch (error) {
console.error('Error fetching stats:', error);
}
}

async function fetchRecentActivity() {
try {
const response = await fetch('/api/stats/recent-activity');
if (!response.ok) throw new Error('Failed to fetch recent activity');
const activities = await response.json();

const activityContainer = document.getElementById("recentActivity");
activityContainer.innerHTML = "";

activities.slice(0, 4).forEach(activity => {
let activityElement;

if (activity.estimatedPrice !== undefined) {
// VEHICLE_OWNER Activity (Shows Booking Details)
activityElement = `
<li class="list-group-item">
<div class="d-flex justify-content-between">
<span>Spot: ${activity.spotName}</span>
<small>Duration: ${activity.duration} hrs</small>
</div>
<p>Amount: $${activity.estimatedPrice}</p>
</li>`;
} else {
// SPOT_OWNER Activity (Shows Spot Details)
activityElement = `
<li class="list-group-item">
<div class="d-flex justify-content-between">
<span>Spot: ${activity.spotName} (${activity.spotType})</span>
<small>Location: ${activity.location}</small>
</div>
<p>Price per Hour: $${activity.pricePerHour}</p>
</li>`;
}

activityContainer.insertAdjacentHTML("beforeend", activityElement);
});
} catch (error) {
console.error('Error fetching recent activity:', error);
```

```
}
}

function updateDashboard() {
fetchStats();
fetchRecentActivity();
}
function updateStats() {
fetch('/api/stats')
.then(response => response.json())
.then(stats => {
if (stats.totalSpots !== undefined) {  // Ensure it's for SPOT_OWNER
document.getElementById('totalSpots').textContent = stats.totalSpots || 0;
document.getElementById('currentBookings').textContent = stats.currentBookings || 0;
document.getElementById('totalRevenue').textContent = `$${stats.totalRevenue || 0}`;
} else {  // VEHICLE_OWNER
document.getElementById('availableSpots').textContent = stats.availableSpots || 0;
document.getElementById('activeBookings').textContent = stats.activeBookings || 0;
document.getElementById('totalHours').textContent = stats.totalHours || 0;
document.getElementById('amountSpent').textContent = `$${stats.amountSpent || 0}`;
}
})
.catch(error => console.error('Error fetching stats:', error));
}

// Update stats every 30 seconds
setInterval(updateStats, 30000);
// Initial update
updateStats();

setInterval(updateDashboard, 30000);
updateDashboard();
</script>

</body>
</html>
```

Backend Code

User Registration module

```
package com.quick_park_assist.dto;

import jakarta.persistence.Column;
import jakarta.validation.constraints.*;

public class UserRegistrationDTO {

@NotBlank(message = "Full name is required")
@Size(min = 2, max = 100, message = "Full name must be between 2 and 100 characters")
private String fullName;
```

```
@NotBlank(message = "Email is required")
@Email(message = "Please provide a valid email address")
@Column(unique = true)
private String email;

@NotBlank(message = "Phone number is required")
@Pattern(regexp = "^\\d{10}$", message = "Please provide a valid 10-digit phone number")
private String phoneNumber;

@NotBlank(message = "User type is required")
private String userType;

@NotBlank(message = "Password is required")
@Size(min = 6, message = "Password must be at least 6 characters long")
@Pattern(regexp = "^(?=.*[0-9])(?=.*[a-zA-Z])(?=.*[@#$%^&+=]).*$", message = "Password must
contain at least one letter, one number, and one special character")
private String password;

@NotBlank(message = "Password confirmation is required")
private String confirmPassword;

@NotBlank(message = "Address is required")
@Size(min = 5, max = 200, message = "Address must be between 5 and 200 characters")
private String address;

// Getters and Setters
public String getFullName() {
return fullName;
}

public void setFullName(String fullName) {
this.fullName = fullName;
}

public String getEmail() {
return email;
}

public void setEmail(String email) {
this.email = email;
}

public String getPhoneNumber() {
return phoneNumber;
}

public void setPhoneNumber(String phoneNumber) {
this.phoneNumber = phoneNumber;
}

public String getUserType() {
return userType;
```

```
}

public void setUserType(String userType) {
this.userType = userType;
}

public String getPassword() {
return password;
}

public void setPassword(String password) {
this.password = password;
}

public String getConfirmPassword() {
return confirmPassword;
}

public void setConfirmPassword(String confirmPassword) {
this.confirmPassword = confirmPassword;
}

public String getAddress() {
return address;
}

public void setAddress(String address) {
this.address = address;
}
}
```

User module

```
package com.quick_park_assist.entity;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "users")
public class User {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   @Column(nullable = false)
```

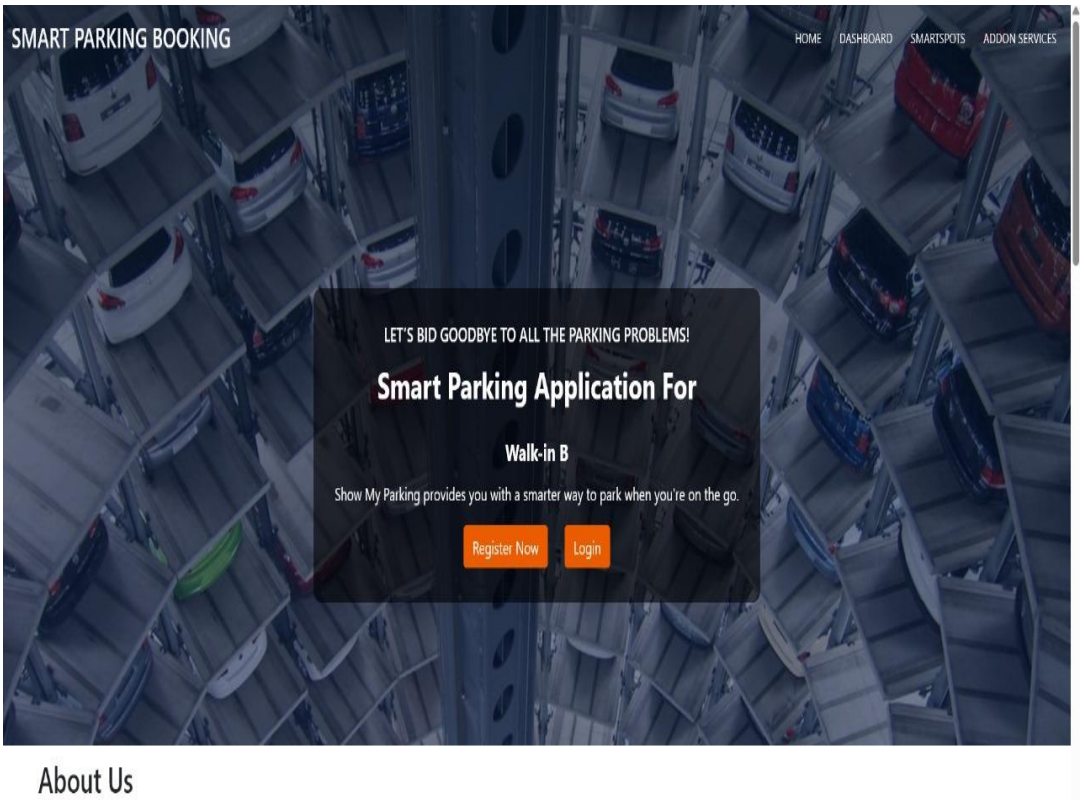
```
    private String fullName;

    @Column(nullable = false, unique = true)
    private String email;

    @NotBlank(message = "Phone number is required")
    @Pattern(regexp = "^\\d{10}$", message = "Please provide a valid 10-digit phone number")

    @Column(nullable = false)
    private String phoneNumber;

    @Column(nullable = false)
    private String userType;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String address;

    @Column(nullable = false)
    private LocalDateTime createdAt;

    @Column(nullable = false)
    private boolean active;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Vehicle> vehicles = new ArrayList<>();

    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getFullName() {
        return fullName;
    }

    public void setFullName(String fullName) {
        this.fullName = fullName;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
```

```
}

public String getPhoneNumber() {
    return phoneNumber;
}

public void setPhoneNumber(String phoneNumber) {
    this.phoneNumber = phoneNumber;
}

public String getUserType() {
    return userType;
}

public void setUserType(String userType) {
    this.userType = userType;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}

public boolean isActive() {
    return active;
}

public void setActive(boolean active) {
    this.active = active;
}


@PrePersist
```

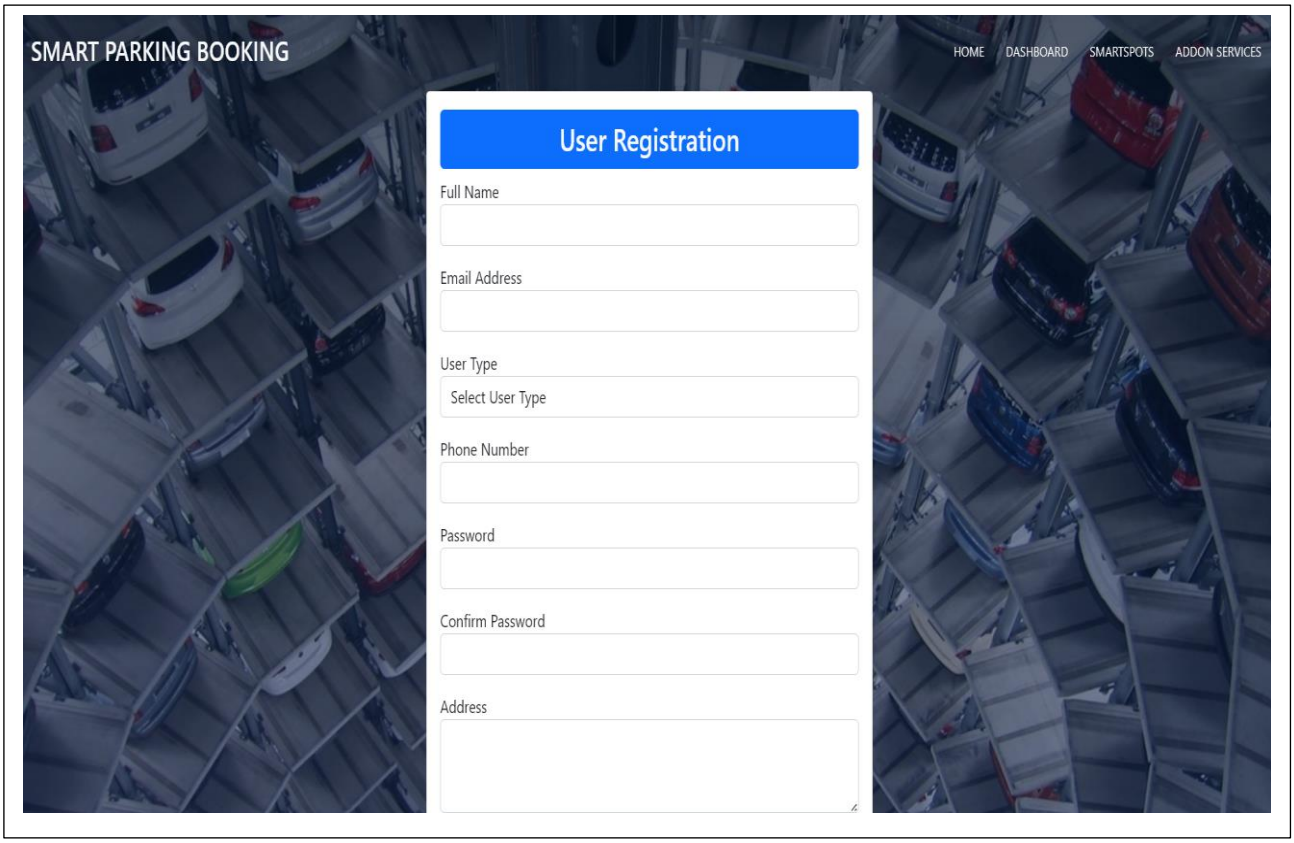
```
    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }


  public void onCreate() {
        createdAt = LocalDateTime.now();
    }

    @Override
    public String toString() {
        return "User{" +
                "id=" + id +
                ", fullName='" + fullName + '\'' +
                ", email='" + email + '\'' +
                ", phoneNumber='" + phoneNumber + '\'' +
                ", userType='" + userType + '\'' +
                ", address='" + address + '\'' +
                ", createdAt=" + createdAt +
                ", active=" + active +
                '}';
    }
}
```

6. OUTPUT

Home Page



User Registration Page



7.TESTING

1. Unit Testing

1.1 Purpose:

Test individual components like controllers, services, and repositories in isolation.

1.2 Tools/Techniques:

- **JUnit:** Java testing framework for writing and executing unit tests.
- **Mockito:** For mocking dependencies and testing components independently.

1.3 What Was Tested:

- Login service
- Booking creation logic
- User registration validation
- Add-on service integration logic

2. Integration Testing

2.1 Purpose:

Ensure that different modules of the application (e.g., user + booking + EV slots) work together as expected.

2.2 Tools/Techniques:

- **Spring Boot Test:** Integration testing using the Spring testing context.

2.3 What Was Tested:

- REST API endpoints working with database and services
- End-to-end test flows like "user registers → books a parking spot → receives confirmation"

3. API Testing

3.1 Purpose:

Verify that the RESTful APIs respond correctly and handle errors gracefully.

3.2 Tools:

- **Postman:** Used to test all HTTP requests and responses manually.
- **Swagger UI:** For testing APIs with documented input/output formats.

3.3 What Was Tested:

- Status codes, response payloads, edge cases
- Authenticated vs. unauthenticated requests
- Valid/invalid data inputs

4. Code Quality & Static Analysis

4.1 Purpose:

Identify code smells, potential bugs, and enforce coding standards.

4.2 Tools:

- **SonarQube:** Scans the codebase and provides metrics on:
  - Code duplication
  - Cyclomatic complexity
  - Security vulnerabilities

5. Log Monitoring and Analysis

5.1 Purpose:

Monitor real-time logs to trace errors, exceptions, and system health.

5.2 Tools:

- **Splunk:** Used to collect and analyze logs for debugging and issue tracking.

6. Performance and Metrics Monitoring

6.1 Purpose:

Track the performance and uptime of APIs.

6.2 Tools:

- **Prometheus + Grafana:**
  - Prometheus collects metrics like request time, server load, DB queries
  - Grafana visualizes trends, spikes, and bottlenecks

8. SWOT ANALYSIS

✅ Strengths

1. Comprehensive solution: Includes parking spot booking, EV charging, and add-on services.
2. User-friendly interface: Aims to provide a seamless experience for both vehicle owners and parking spot providers.
3. Technology Stack: Uses robust technologies like Java, Spring Boot, and MySQL.

⚠️ Weaknesses

1. Lack of details: The presentation lacks detailed information on database design, testing procedures, and cost estimation.
2. No mobile app: The presentation mentions it as a future enhancement, but the current version lacks a dedicated mobile application.

🌱 Opportunities

1. Future Enhancements: AI-based dynamic pricing, mobile application, and IoT-based real-time parking availability.
2. Growing Market: Increasing number of vehicles and EVs creates a demand for smart parking solutions.

🚫 Threats

1. Competition: Existing parking apps and traditional parking management systems.
2. Adoption Rate: Success depends on user adoption by both vehicle owners and parking spot providers.

9. CONCLUSION

Quick Park Assist is an innovative technology designed to simplify the parking process, offering increased convenience, safety, and efficiency. By utilizing sensors, cameras, and advanced algorithms, it helps drivers park their vehicles with ease, reducing the likelihood of collisions and minimizing the stress often associated with parking in tight spaces. This system enhances the overall driving experience, particularly in urban environments where parking can be a challenge. With its intuitive interface and real-time feedback, Quick Park Assist is a valuable tool for both novice and experienced drivers. As technology continues to evolve, it holds the potential to integrate even further with autonomous driving systems, making parking an entirely hands-free process in the future.

10. REFERENCE

[1] M. Idris, Y. Leng, E. Tamil, N. Noor, and Z. Razak, "Car Park System: A Review of Smart Parking System and its Technology," Information Technology Journal, vol. 8, no. 2, pp. 101–113, 2009.
https://doi.org/10.3923/itj.2009.101.113

[2] M. Khanna and R. Anand, "IoT based Smart Parking System," Proc. IEEE IOTA, 2016.
https://doi.org/10.1109/IOTA.2016.7562735

[3] A. Sharma et al., "Automated Car Parking using IoT," Proc. IEEE SEISCON, 2018.
https://doi.org/10.1109/SEISCON.2018.8771402

[4] Y. Kamarudin et al., "Smart Parking Reservation System," IJACSA, vol. 10, no. 8, pp. 1–6, 2019.
https://doi.org/10.14569/IJACSA.2019.0100861

[5] R. Shah et al., "RBAC System for Cloud Web Apps," Proc. IEEE CICN, 2015.
https://doi.org/10.1109/CICN.2015.59

[6] Thymeleaf, "Java template engine," 2025.
https://www.thymeleaf.org

[7] Spring Boot, "Reference Documentation," 2025.
https://docs.spring.io/spring-boot

[8] Prometheus, "Monitoring System Docs," 2025.
https://prometheus.io/docs

[9] Grafana Labs, "Grafana Docs," 2025.
https://grafana.com/docs

[10] Postman, "API Platform Docs," 2025.
https://www.postman.com